LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Muhammad Khairul Anaam 2609106026

Kelas A2‘26

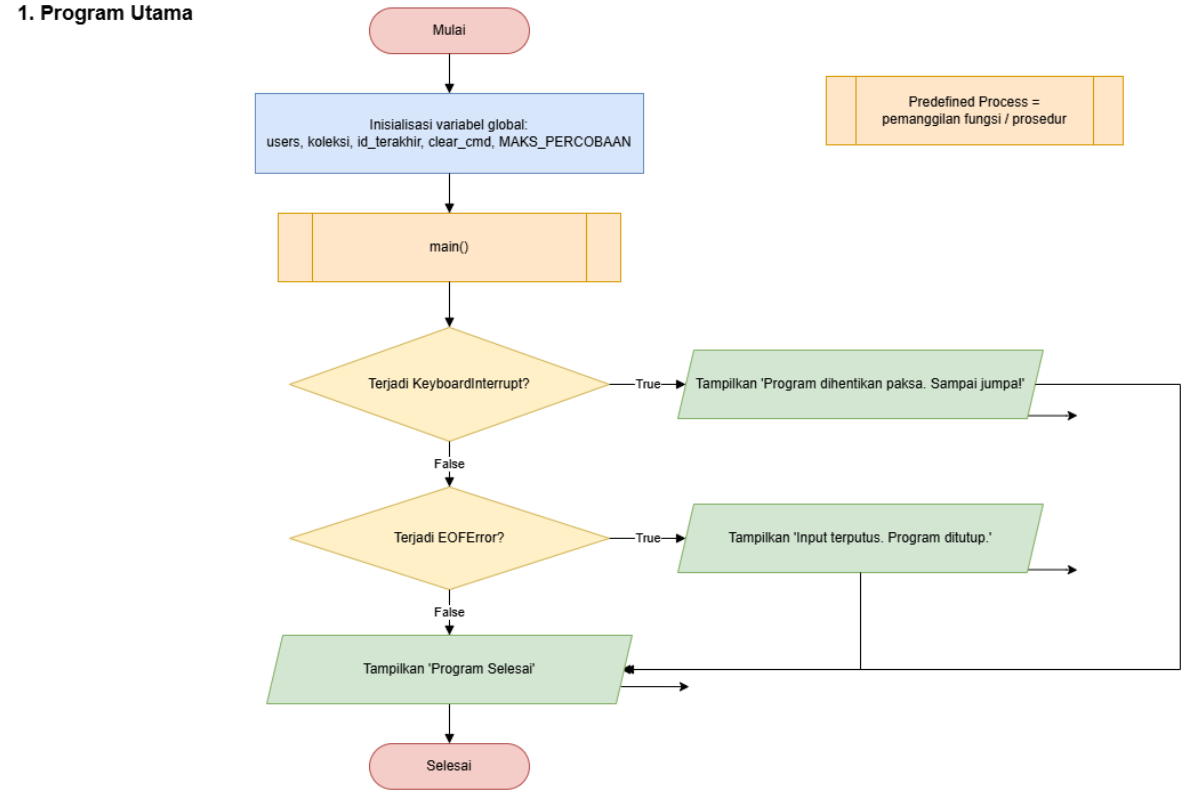
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart

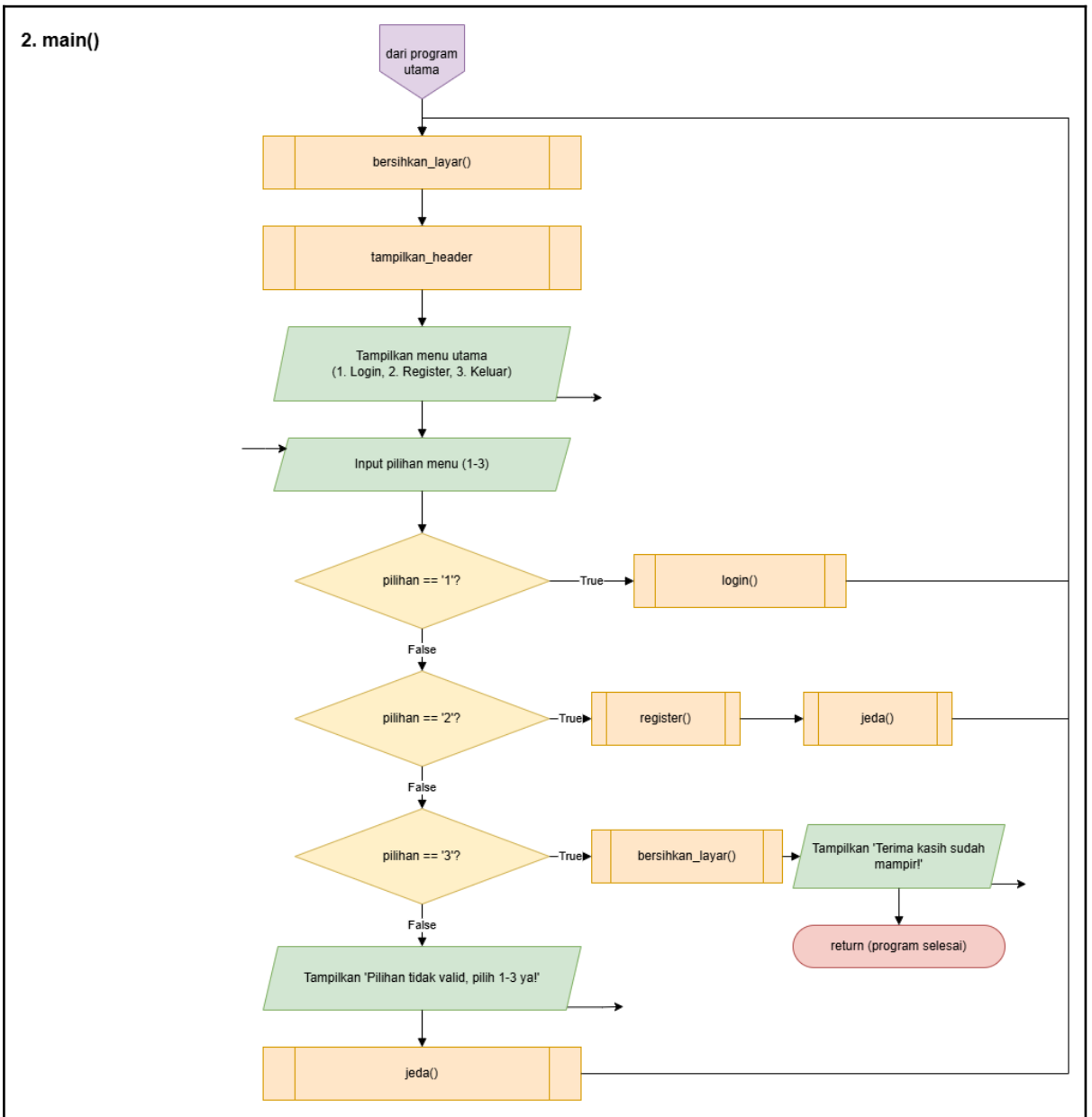


Gambar 1.1 Program Utama

Flowchart ini menggambarkan alur program utama pada program Koleksi Gantungan Kunci. Program dimulai dengan menyiapkan variabel global, yaitu users, koleksi, id_terakhir, clear_cmd, dan MAKS_PERCOBAAN. Setelah itu program memanggil fungsi main(), yang digambarkan dengan simbol Predefined Process karena main() adalah fungsi yang prosesnya dijelaskan secara terpisah. Di dalam main() terdapat menu utama (Login, Register, Keluar) beserta seluruh fitur program, yang dijabarkan pada halaman flowchart berikutnya.

Pemanggilan main() dibungkus dengan error handling (try/except/finally), sehingga program tidak langsung crash jika terjadi kesalahan. Program mengecek dua jenis error, mulai dari KeyboardInterrupt. Jika pengguna menekan Ctrl+C (True), program menampilkan pesan “Program dihentikan paksa. Sampai jumpa!”. Jika tidak (False), program mengecek EOFError. Jika input terputus (True), program menampilkan pesan “Input terputus. Program ditutup.”.

Apapun yang terjadi, baik terjadi error maupun tidak, program selalu menjalankan blok finally. Pada flowchart ini blok finally digambarkan sebagai proses menampilkan “Program Selesai”. Setelah itu program berakhir pada simbol Selesai.



Gambar 1.2 Menu Main

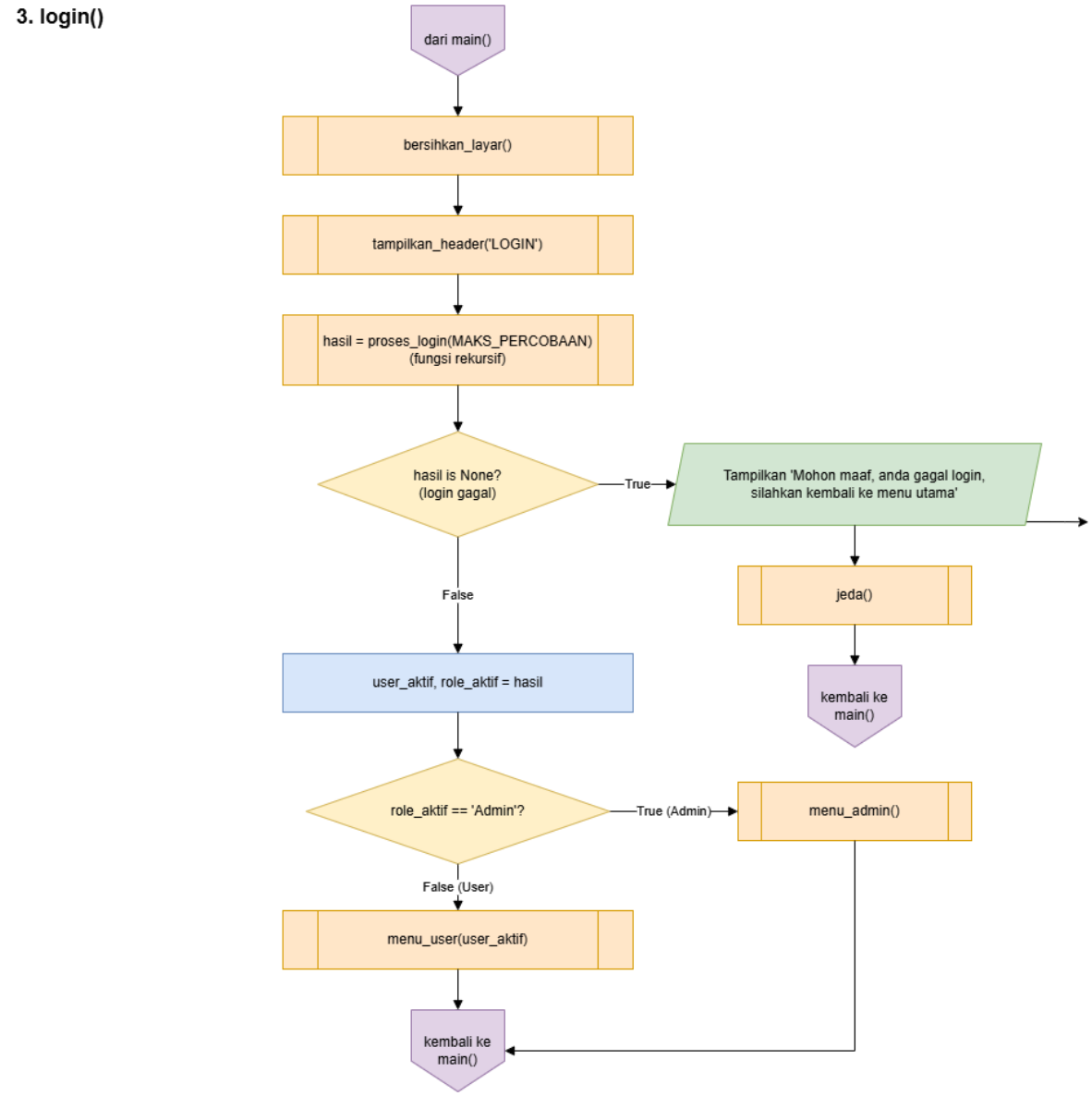
Flowchart ini menggambarkan alur fungsi main(), yaitu menu utama pada program Koleksi Gantungan Kunci. Alur dimulai dari konektor “dari program utama”, lalu program memanggil prosedur bersihkan_layar() dan tampilkan_header() untuk membersihkan layar dan mencetak judul program. Setelah itu program menampilkan menu utama, yaitu 1. Login,

2. Register, dan 3. Keluar, lalu pengguna diminta memasukkan pilihan menu (1-3). Semua pemanggilan fungsi dan prosedur digambarkan dengan simbol Predefined Process (persegi panjang bergaris ganda).

Pilihan pengguna dicek satu per satu.

- Jika pengguna memilih 1, program memanggil fungsi login(). Setelah proses login selesai, program kembali ke awal main() dan menampilkan menu utama lagi.
- Jika memilih 2, program memanggil register() untuk membuat akun baru, lalu memanggil jeda() agar pengguna sempat membaca hasilnya. Setelah itu program kembali ke menu utama.
- Jika memilih 3, program memanggil bersihkan_layar(), menampilkan pesan “Terima kasih sudah mampir!”, lalu main() berakhir dengan return, sehingga program selesai.
- Jika pilihan tidak sesuai dengan ketiganya, program menampilkan pesan “Pilihan tidak valid, pilih 1-3 ya!”, memanggil jeda(), lalu kembali ke awal untuk menampilkan menu utama lagi.

Perulangan menu digambarkan dengan garis panah di sisi kanan yang kembali ke atas, tepat sebelum bersihkan_layar(). Cara ini menggantikan variabel program_jalan pada APD 6. Perulangan berhenti hanya pada pilihan 3, yaitu saat main() mengakhiri dirinya dengan return.



Gambar 1.3 Menu login

Flowchart ini menggambarkan alur fungsi login(), yaitu proses masuk ke akun pada program Koleksi Gantungan Kunci. Alur dimulai dari konektor “dari main()”, lalu program memanggil prosedur bersihkan_layar() dan tampilkan_header('LOGIN') untuk membersihkan layar dan menampilkan judul halaman login. Setelah itu program memanggil fungsi proses_login(MAKS_PERCOBAAN) dan menyimpan hasilnya ke variabel hasil. Fungsi ini bersifat rekursif, yaitu memanggil dirinya sendiri sampai login berhasil atau percobaan habis (maksimal 3 kali). Detail prosesnya digambarkan pada flowchart proses_login() di halaman berikutnya.

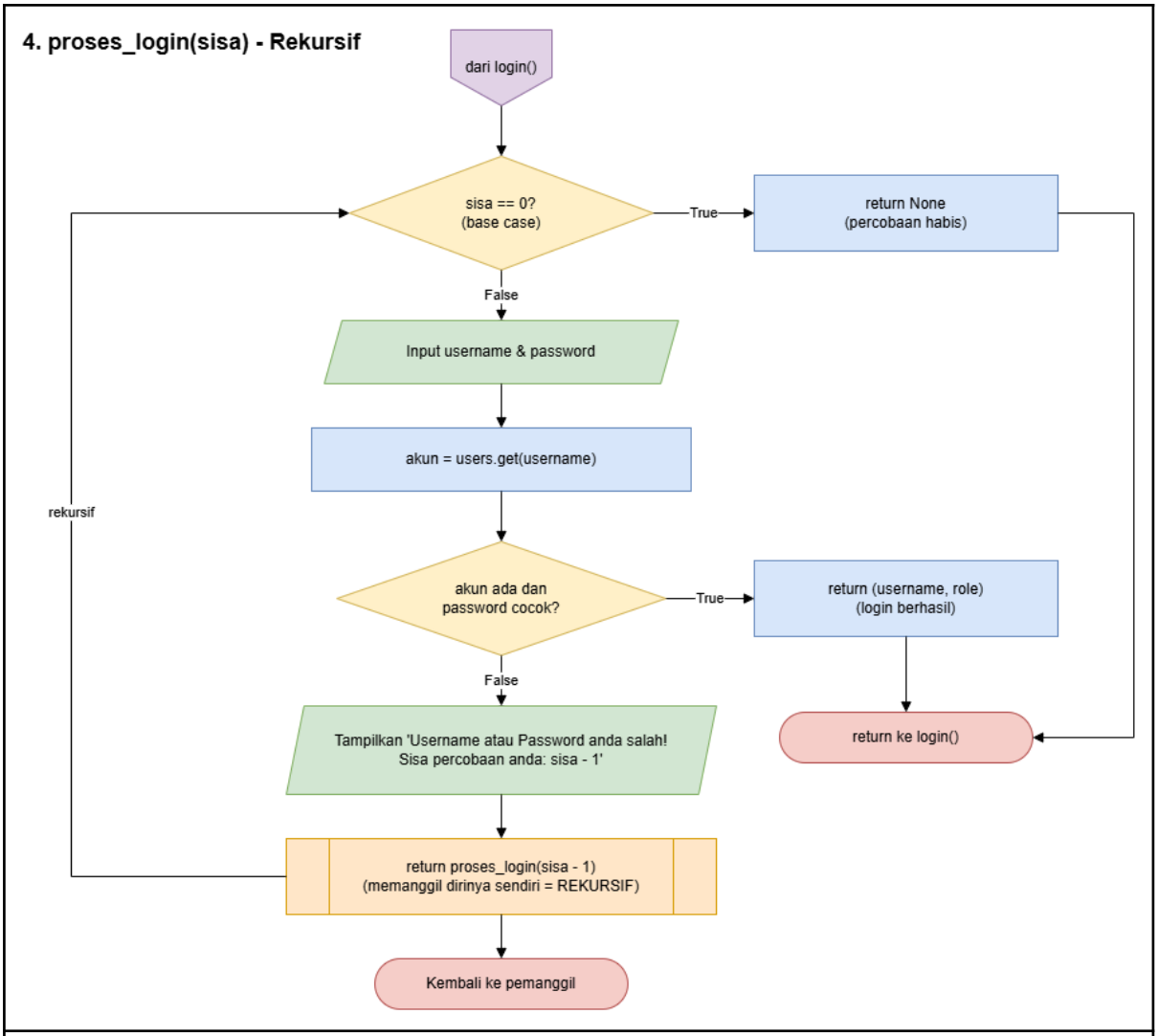
Selanjutnya program mengecek apakah hasil bernilai None.

- Jika True, berarti login gagal karena percobaan sudah habis. Program menampilkan pesan “Mohon maaf, anda gagal login, silahkan kembali ke menu utama”, memanggil jeda(), lalu kembali ke main() melalui konektor “kembali ke main()”.
- Jika False, berarti login berhasil. Data dalam hasil dipecah menjadi user_aktif dan role_aktif. Setelah itu program mengecek apakah role_aktif bernilai ‘Admin’.

Pengecekan role inilah yang membedakan dua jenis pengguna (multiuser).

- Jika role adalah Admin (True), program memanggil menu_admin(), yang berisi fitur tambah, lihat, ubah, dan hapus koleksi.
- Jika bukan Admin (False), program memanggil menu_user(user_aktif), yang hanya berisi fitur melihat koleksi.

Setelah pengguna logout dari menu masing-masing, program kembali ke main() melalui konektor “kembali ke main()”.



Gambar 1.4 Proses Login

Flowchart ini menggambarkan alur fungsi rekursif proses_login(sisa), yaitu proses pengecekan username dan password yang diulang secara otomatis sampai login berhasil atau percobaan habis. Alur dimulai dari konektor “dari login()”. Parameter sisa berisi jumlah percobaan yang tersisa, dan nilai awalnya adalah MAKS_PERCOBAAN, yaitu 3.

Pertama, program mengecek base case, yaitu apakah sisa == 0. Base case adalah kondisi yang menghentikan rekursi.

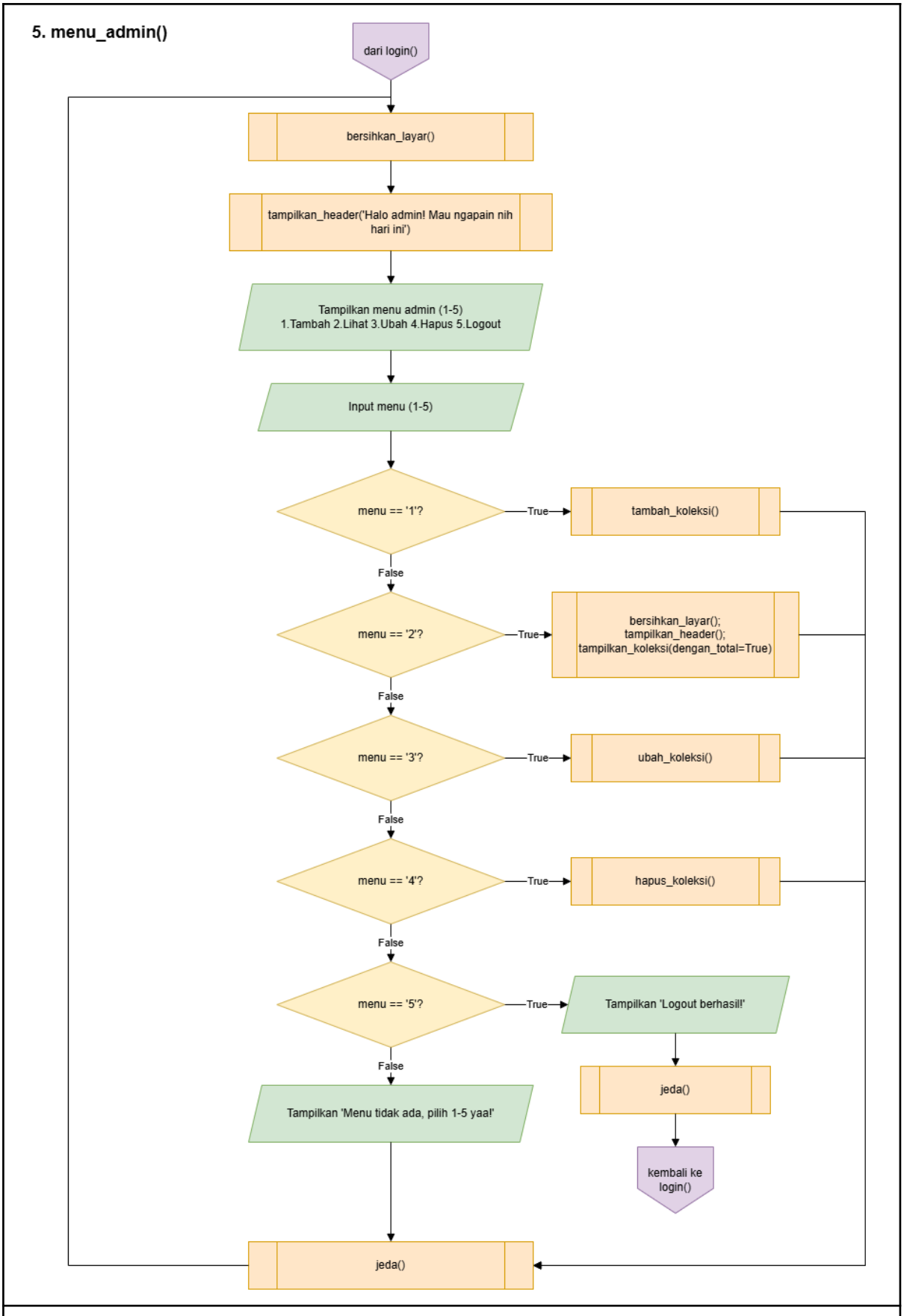
- Jika True, percobaan sudah habis. Fungsi mengembalikan None (login gagal), lalu kembali ke login().
- Jika False, program meminta pengguna memasukkan username dan password. Setelah itu program mengambil data akun dengan users.get(username), lalu mengecek apakah akun ada dan password-nya cocok.

Hasil pengecekan akun menentukan langkah berikutnya.

- Jika cocok (True), login berhasil. Fungsi mengembalikan (username, role) lalu kembali ke login(), dan rekursi berhenti.
- Jika tidak cocok (False), program menampilkan pesan “Username atau Password anda salah!” beserta sisa percobaan, yaitu sisa - 1. Setelah itu fungsi memanggil dirinya sendiri dengan return proses_login(sisa - 1).

Pemanggilan diri sendiri ini adalah rekursif recursive case, digambarkan dengan simbol Predefined Process dan panah kembali ke pengecekan sisa == 0. Setiap pemanggilan membuat nilai sisa berkurang satu, sehingga lama-kelamaan mendekati base case dan rekursi pasti berhenti. Hasil pemanggilan terdalam kemudian dikembalikan ke pemanggil sebelumnya hingga sampai ke login().

Sebagai contoh, jika pengguna salah memasukkan password dua kali lalu benar pada percobaan ketiga, urutannya adalah proses_login(3), proses_login(2), dan proses_login(1). Pada pemanggilan terakhir akunnya cocok, sehingga mengembalikan (username, role).



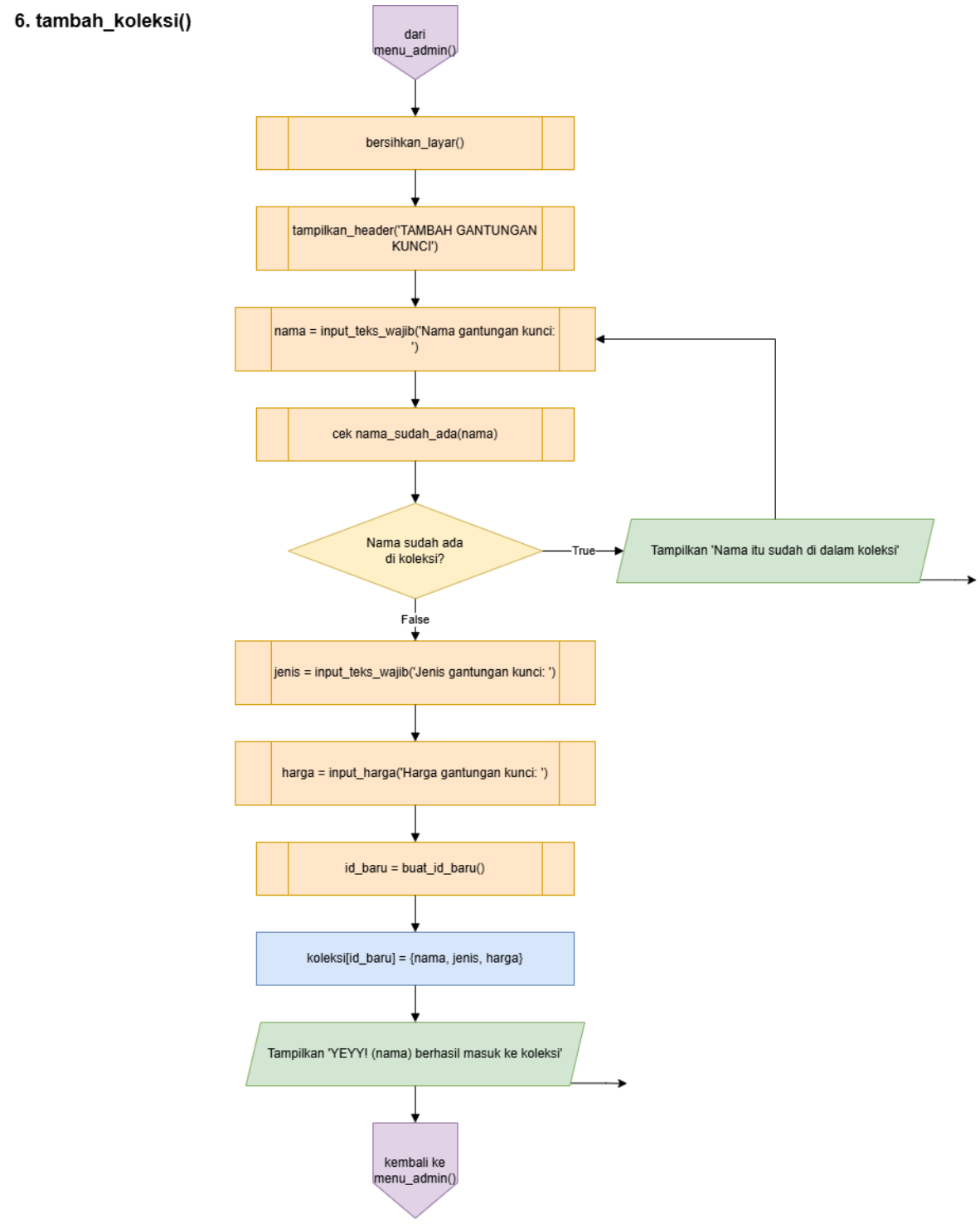
Gambar 1.5 Menu Admin

Flowchart ini menggambarkan alur prosedur menu_admin(), yaitu menu khusus untuk Admin setelah berhasil login. Alur dimulai dari konektor “dari login()”. Program memanggil bersihkan_layar() dan tampilkan_header('Halo admin! Mau ngapain nih hari ini'), lalu menampilkan menu admin (1. Tambah, 2. Lihat, 3. Ubah, 4. Hapus, 5. Logout). Setelah itu pengguna diminta memasukkan pilihan menu (1-5).

Pilihan tersebut dicek satu per satu, dan setiap fitur dipanggil sebagai prosedur terpisah (simbol Predefined Process).

- Jika memilih 1, program memanggil tambah_koleksi() untuk menambahkan gantungan kunci baru.
- Jika memilih 2, program memanggil bersihkan_layar(), tampilkan_header(), dan tampilkan_koleksi(dengan_total=True) untuk menampilkan seluruh koleksi beserta total jumlah dan total nilainya.
- Jika memilih 3, program memanggil ubah_koleksi() untuk mengubah data koleksi.
- Jika memilih 4, program memanggil hapus_koleksi() untuk menghapus koleksi.
- Jika memilih 5, program menampilkan pesan “Logout berhasil!”, memanggil jeda(), lalu kembali ke login() melalui konektor “kembali ke login()”. Pilihan ini menghentikan perulangan menu admin.
- Jika pilihan tidak sesuai dengan kelimanya, program menampilkan pesan “Menu tidak ada, pilih 1-5 yaa!”.

Setelah fitur 1 sampai 4 selesai, atau setelah muncul pesan menu tidak valid, program memanggil jeda() agar pengguna sempat membaca hasilnya. Setelah itu program kembali ke awal, yaitu bersihkan_layar(), sehingga menu admin muncul lagi. Perulangan ini digambarkan dengan garis panah di sisi kiri yang naik kembali ke atas. Perulangan hanya berhenti saat admin memilih 5 (Logout).



Gambar 1.6 Tambah Koleksi

Flowchart ini menggambarkan alur prosedur tambah_koleksi(), yaitu fitur Admin untuk menambahkan gantungan kunci baru ke dalam koleksi. Alur dimulai dari konektor "dari menu_admin()". Program memanggil bersihkan_layar() dan

tampilkan_header('TAMBAH GANTUNGAN KUNCI') untuk membersihkan layar dan menampilkan judul halaman.

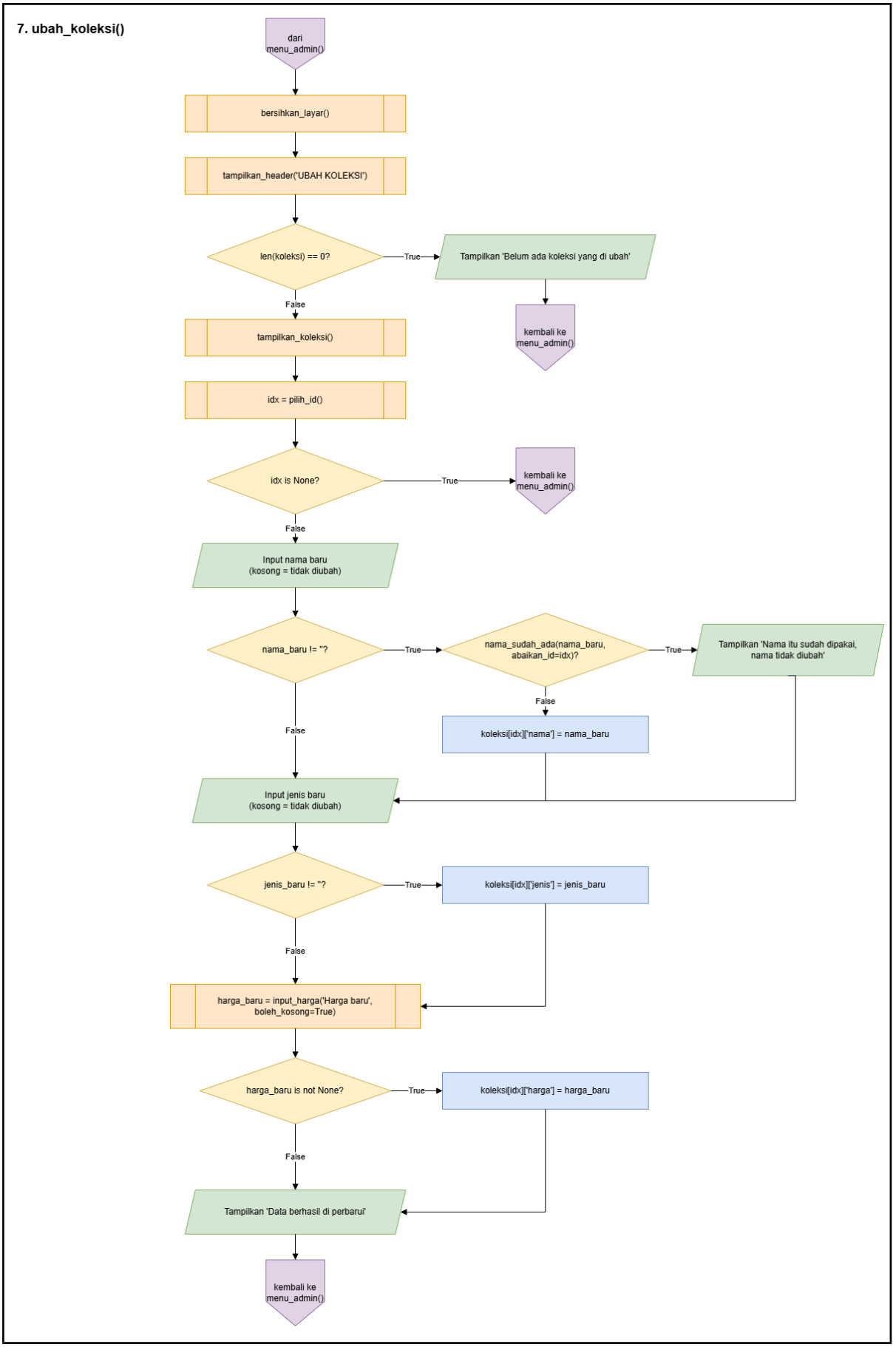
Setelah itu program meminta nama gantungan kunci dengan fungsi input_teks_wajib('Nama gantungan kunci: '). Fungsi ini memastikan nama tidak boleh kosong. Nama yang diinput lalu dicek dengan fungsi nama_sudah_ada(nama) untuk melihat apakah nama tersebut sudah ada di koleksi (tanpa membedakan huruf besar dan kecil).

- Jika nama sudah ada (True), program menampilkan pesan “Nama itu sudah di dalam koleksi”, lalu kembali ke input nama sampai pengguna memasukkan nama yang belum terpakai.
- Jika nama belum ada (False), program lanjut ke langkah berikutnya.

Selanjutnya program meminta jenis gantungan kunci dengan input_teks_wajib('Jenis gantungan kunci: ') dan harga dengan input_harga('Harga gantungan kunci: '). Fungsi input_harga() memastikan harga berupa angka bulat lebih dari 0, dan input yang salah ditangani dengan error handling (try/except). Setelah semua data valid, program memanggil fungsi buat_id_baru() untuk mendapatkan ID baru, yang menaikkan variabel global id_terakhir.

Data nama, jenis, dan harga kemudian disimpan ke dictionary koleksi dengan ID baru tersebut. Terakhir, program menampilkan pesan “YEYY! (nama) berhasil masuk ke koleksi” dan kembali ke menu_admin() melalui konektor “kembali ke menu_admin()”.

7. ubah_koleksi()



Gambar 1.7 Ubah Koleksi

Flowchart ini menggambarkan alur prosedur ubah_koleksi(), yaitu fitur Admin untuk mengubah data gantungan kunci yang sudah ada di koleksi. Alur dimulai dari konektor “dari menu_admin()”. Program memanggil bersihkan_layar() dan tampilkan_header('UBAH KOLEKSI'), lalu mengecek apakah koleksi kosong (len(koleksi) == 0).

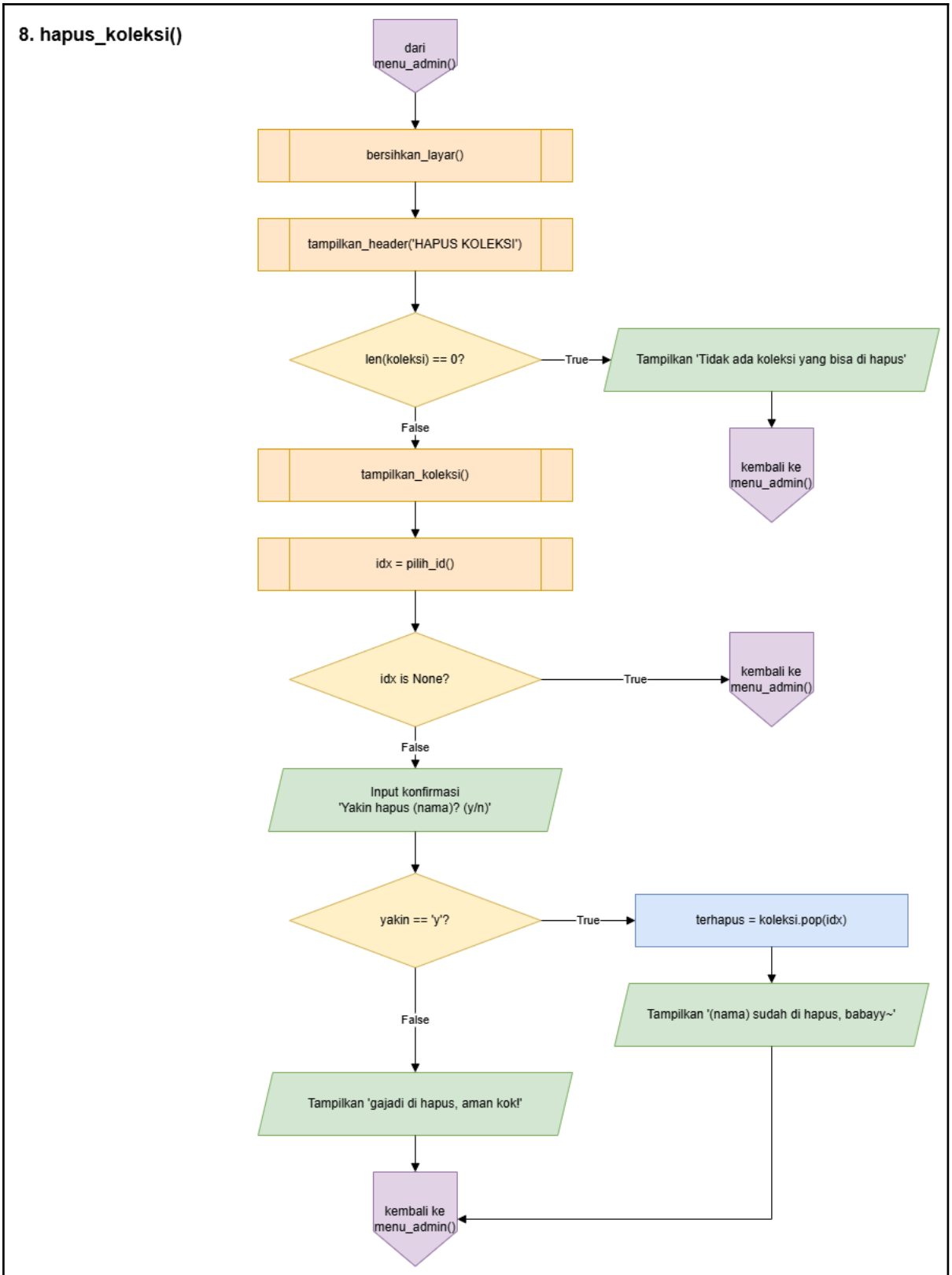
- Jika True, program menampilkan pesan “Belum ada koleksi yang di ubah”, lalu kembali ke menu_admin().
- Jika False, program memanggil tampilkan_koleksi() untuk menampilkan daftar koleksi, lalu memanggil fungsi pilih_id() dan menyimpan hasilnya ke variabel idx.

Fungsi pilih_id() memakai error handling, sehingga ID yang bukan angka atau tidak ada di koleksi tidak membuat program crash, melainkan mengembalikan None. Karena itu program mengecek idx is None. Jika True, ID tidak valid dan program langsung kembali ke menu_admin(). Jika False, proses pengubahan dilanjutkan.

Proses pengubahan dilakukan per bagian, dan setiap bagian boleh dikosongkan jika tidak ingin diubah.

- Nama: program meminta nama baru. Jika diisi (nama_baru != ''), program mengecek dengan nama_sudah_ada(nama_baru, abaikan_id=idx) apakah nama tersebut sudah dipakai item lain. Jika sudah dipakai, program menampilkan pesan “Nama itu sudah dipakai, nama tidak diubah”. Jika belum, nama diperbarui dengan koleksi[idx]['nama'] = nama_baru. Jika dikosongkan, bagian ini dilewati.
- Jenis: program meminta jenis baru. Jika diisi (jenis_baru != ''), jenis diperbarui dengan koleksi[idx]['jenis'] = jenis_baru. Jika dikosongkan, bagian ini dilewati.
- Harga: program memanggil input_harga('Harga baru', boleh_kosong=True). Jika hasilnya bukan None (harga valid), harga diperbarui dengan koleksi[idx]['harga'] = harga_baru. Jika None, berarti dikosongkan dan harga tidak diubah.

Setelah semua bagian diproses, program menampilkan pesan “Data berhasil di perbarui” dan kembali ke menu_admin() melalui konektor “kembali ke menu_admin()”.



Gambar 1.8 Hapus Koleksi

Flowchart ini menggambarkan alur prosedur hapus_koleksi(), yaitu fitur Admin untuk menghapus gantungan kunci dari koleksi. Alur dimulai dari konektor “dari menu_admin()”.

Program memanggil bersihkan_layar() dan tampilkan_header('HAPUS KOLEKSI'), lalu mengecek apakah koleksi kosong (len(koleksi) == 0).

- Jika True, program menampilkan pesan “Tidak ada koleksi yang bisa di hapus”, lalu kembali ke menu_admin().
- Jika False, program memanggil tampilkan_koleksi() untuk menampilkan daftar koleksi, lalu memanggil fungsi pilih_id() dan menyimpan hasilnya ke variabel idx.

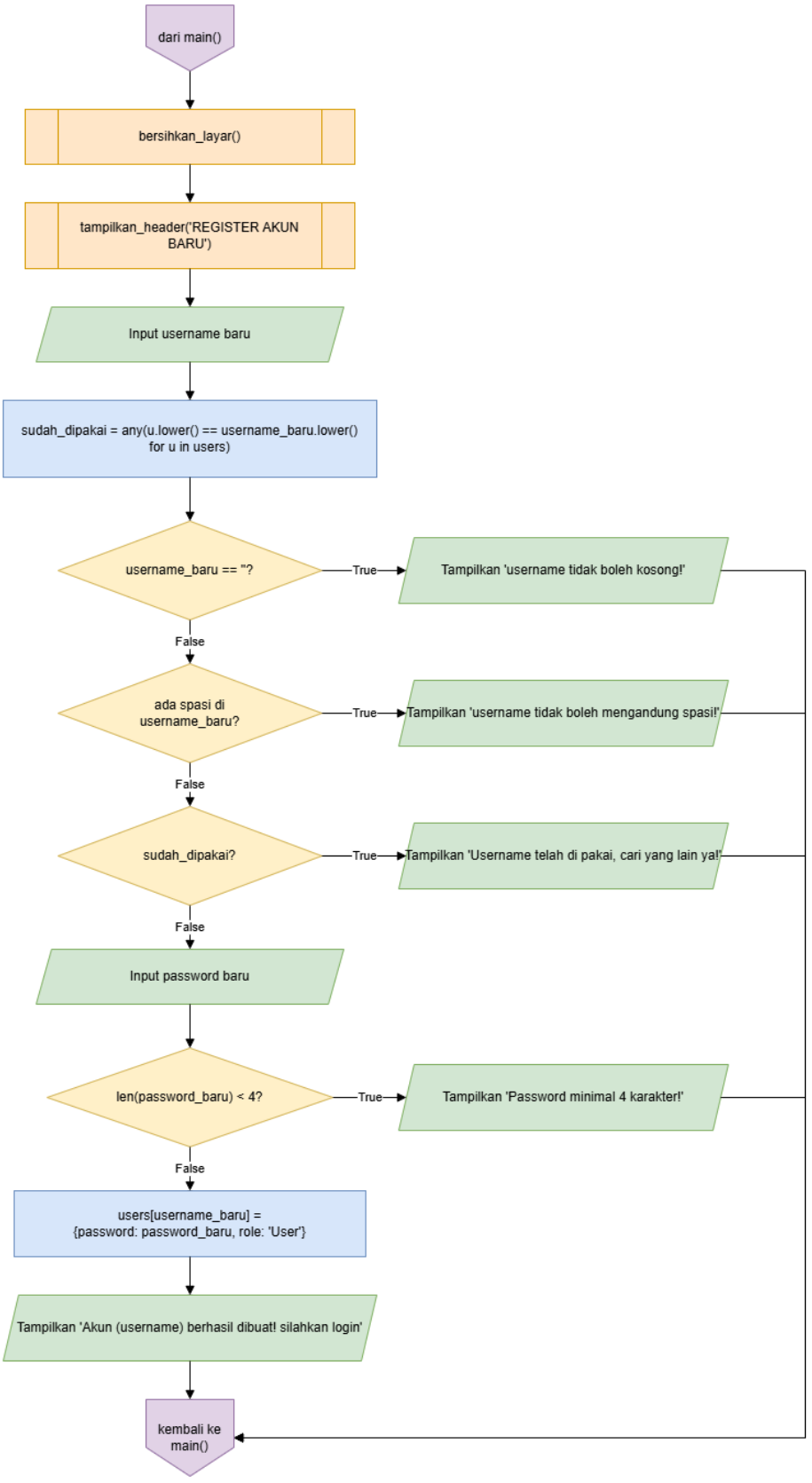
Fungsi pilih_id() memakai error handling, sehingga ID yang bukan angka atau tidak ada di koleksi tidak membuat program crash, melainkan mengembalikan None. Karena itu program mengecek idx is None. Jika True, ID tidak valid dan program langsung kembali ke menu_admin(). Jika False, program lanjut ke konfirmasi.

Pada tahap konfirmasi, program menampilkan pertanyaan “Yakin hapus (nama)? (y/n)” dan meminta pengguna memasukkan jawaban, lalu mengecek apakah yakin == 'y'.

- Jika True, program menghapus data dengan koleksi.pop(idx), menyimpannya ke variabel terhapus, lalu menampilkan pesan “(nama) sudah di hapus, babayy~”.
- Jika False, data tidak dihapus dan program menampilkan pesan “gajadi di hapus, aman kok!”.

Kedua jalur akhirnya bertemu di konektor “kembali ke menu_admin()”, sehingga program kembali ke menu admin.

9. register()

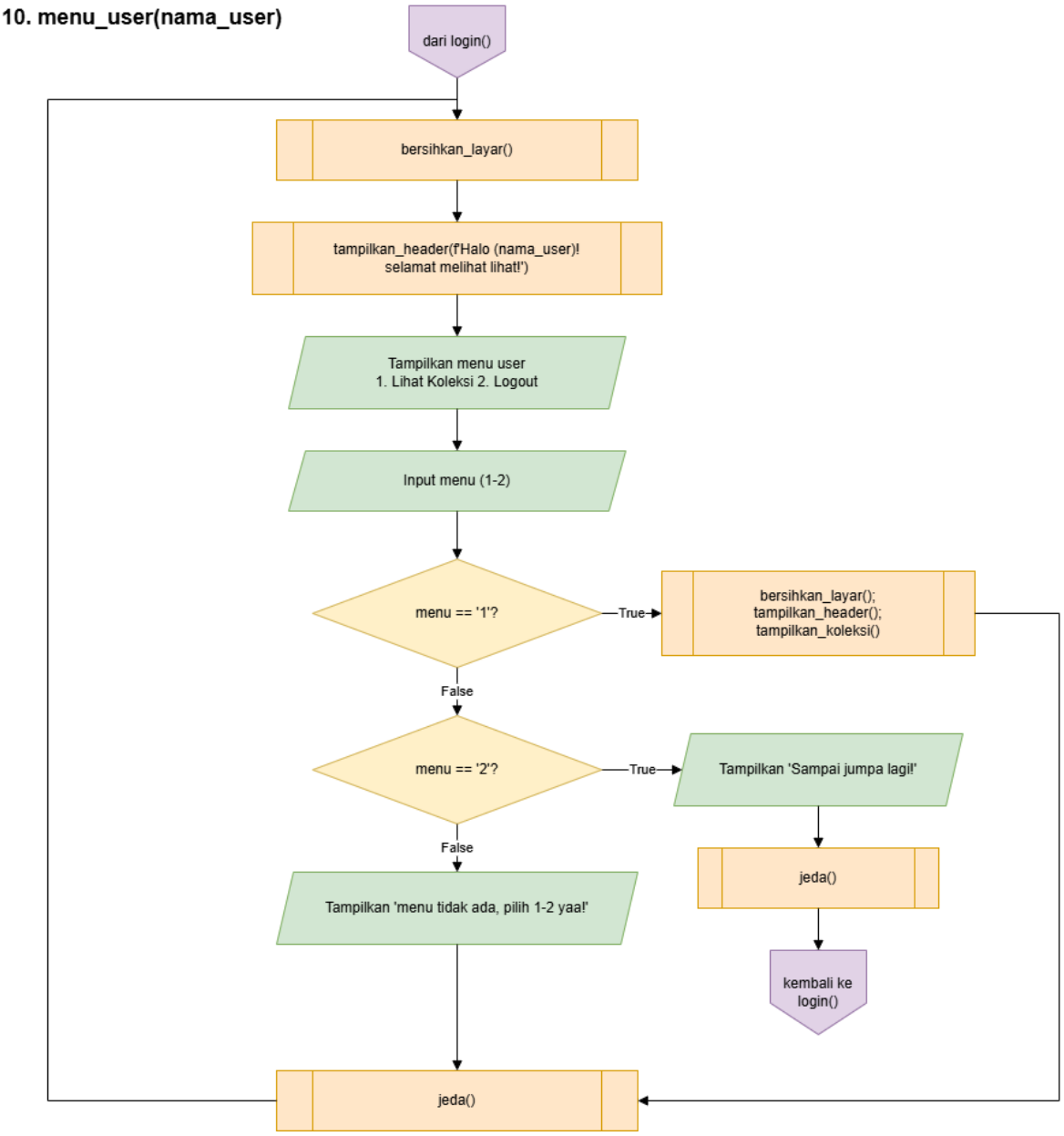


Gambar 1.9 Register Akun baru

Flowchart ini menggambarkan alur prosedur register(), yaitu pembuatan akun baru oleh pengguna. Alur dimulai dari konektor “dari main()”. Program memanggil bersihkan_layar() dan tampilkan_header('REGISTER AKUN BARU'), lalu meminta username baru. Program juga mengecek apakah username sudah dipakai dengan any(u.lower() == username_baru.lower() for u in users), dan hasilnya disimpan di sudah_dipakai. Setelah itu username divalidasi bertahap.

- Jika kosong, tampil pesan “username tidak boleh kosong!”.
- Jika mengandung spasi, tampil pesan “username tidak boleh mengandung spasi!”.
- Jika sudah dipakai, tampil pesan “Username telah di pakai, cari yang lain ya!”.

Jika username lolos ketiga pengecekan, program meminta password baru. Jika panjang password kurang dari 4 karakter, tampil pesan “Password minimal 4 karakter!”. Jika valid, akun disimpan ke users dengan role ‘User’, lalu tampil pesan “Akun berhasil dibuat! silahkan login”. Semua jalur, baik gagal maupun berhasil, berakhir di konektor “kembali ke main()”.



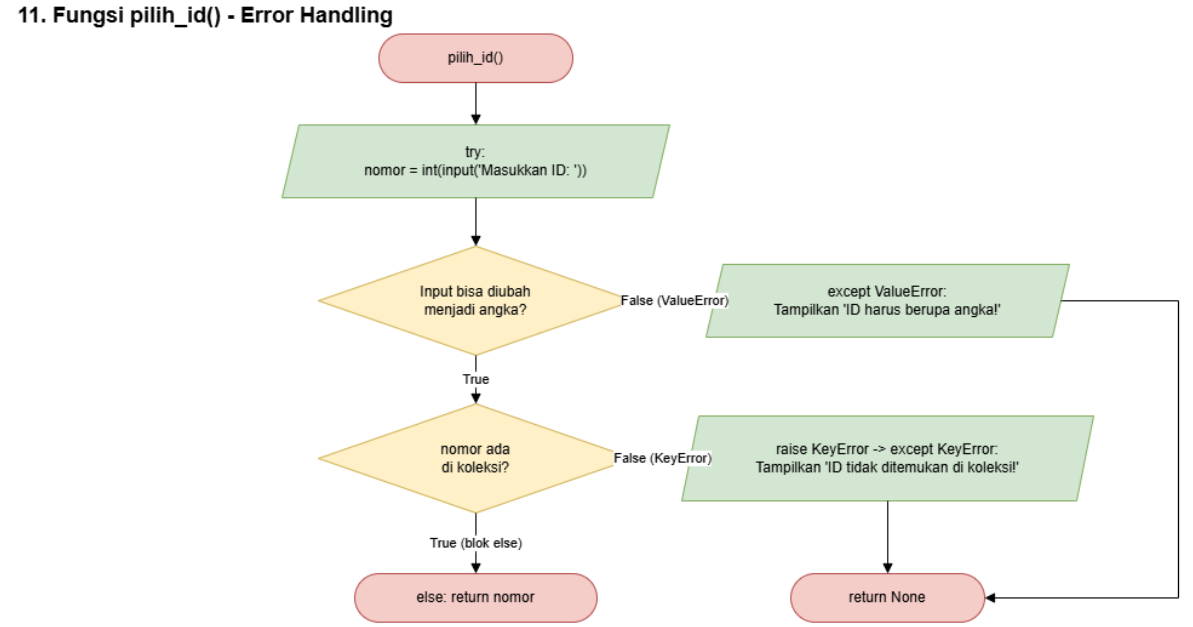
Gambar 1.10 Menu User

Flowchart ini menggambarkan alur prosedur menu_user(), yaitu menu untuk pengguna biasa setelah login. Program memanggil bersihkan_layar() dan tampilkan_header() dengan sapaan sesuai nama_user, lalu menampilkan menu (1. Lihat Koleksi, 2. Logout) dan meminta pilihan (1-2).

- Jika memilih 1, program memanggil bersihkan_layar(), tampilkan_header(), dan tampilkan_koleksi() untuk menampilkan daftar koleksi.
- Jika memilih 2, program menampilkan “Sampai jumpa lagi!”, memanggil jeda(), lalu kembali ke login(). Pilihan ini menghentikan perulangan.

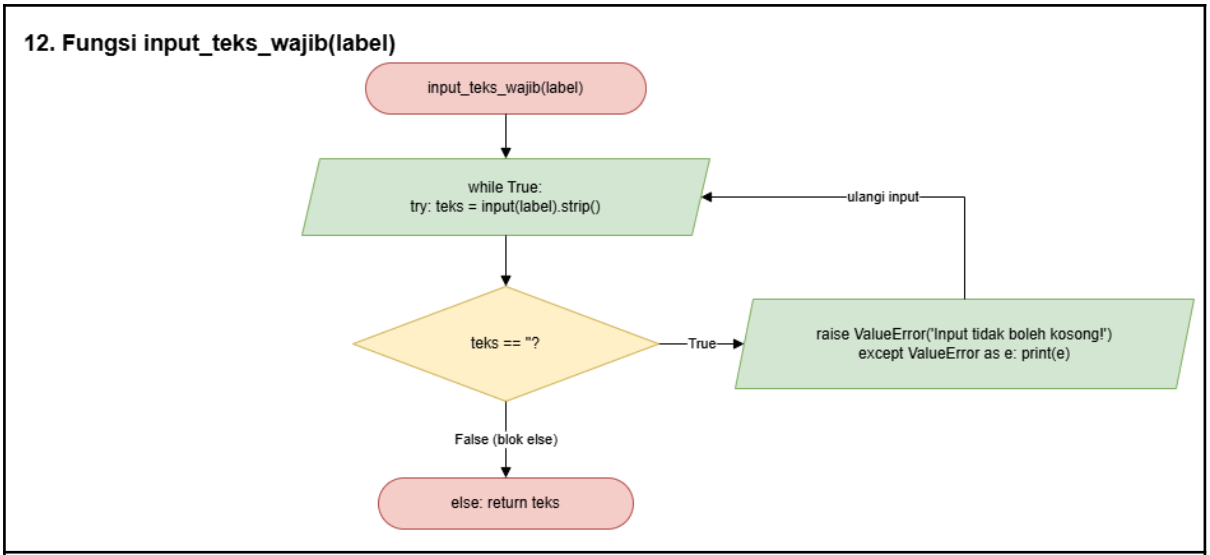
- Jika pilihan lain, tampil pesan “menu tidak ada, pilih 1-2 yaa!”.

Setelah pilihan 1 atau pilihan tidak valid, program memanggil jeda() lalu kembali ke awal untuk menampilkan menu lagi. Berbeda dengan Admin, pengguna biasa hanya bisa melihat koleksi dan tidak bisa menambah, mengubah, atau menghapus. Inilah perbedaan hak akses pada fitur multiuser.



Gambar 1.11 Fungsi Pilih ID

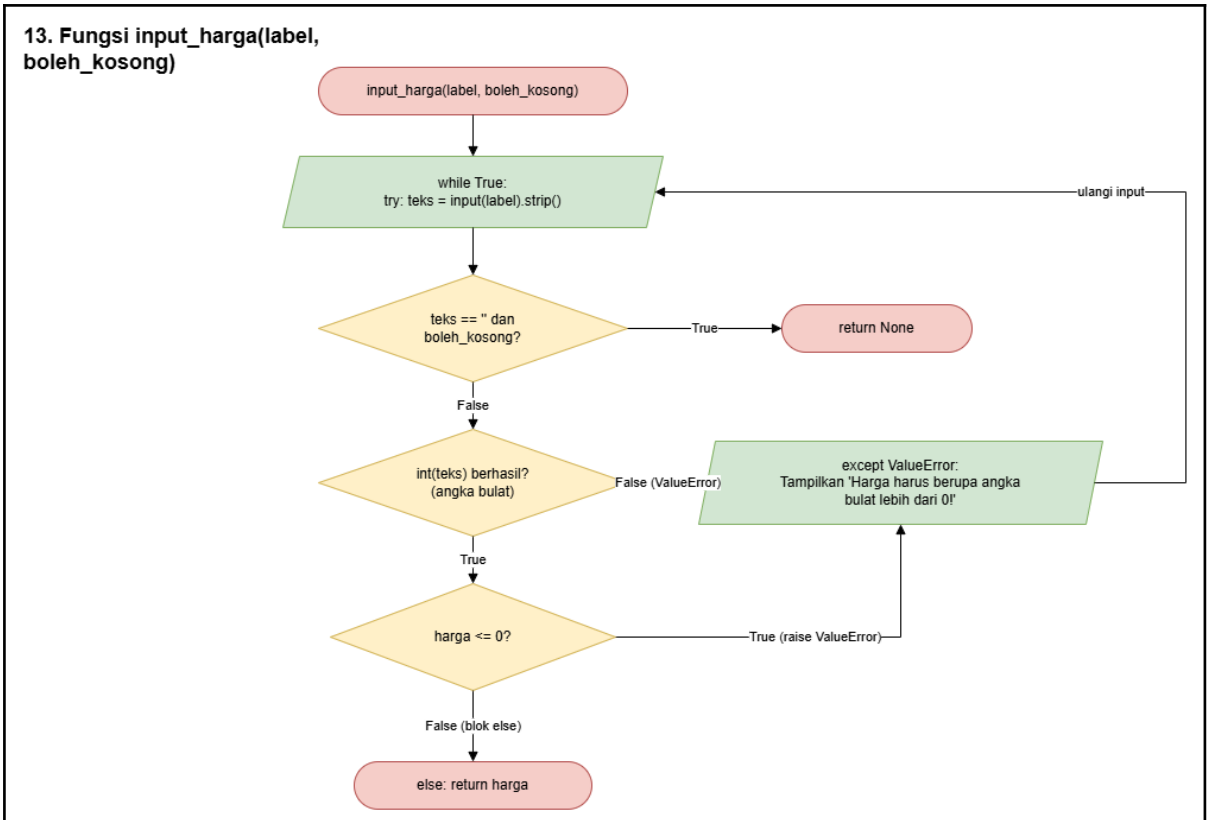
Flowchart ini menggambarkan fungsi tanpa parameter pilih_id(), yang meminta ID dari pengguna dengan error handling. Di dalam blok try, program mengubah input menjadi angka dengan int(input(...)). Jika gagal diubah (False), terjadi ValueError, sehingga except ValueError menampilkan “ID harus berupa angka!” lalu mengembalikan None. Jika berhasil (True), program mengecek apakah nomor ada di koleksi. Jika tidak ada (False), program melakukan raise KeyError, dan except KeyError menampilkan “ID tidak ditemukan di koleksi!” lalu mengembalikan None. Jika ada (True), blok else dijalankan dan fungsi mengembalikan nomor.



Gambar 1.12 Fungsi Input Teks Wajib

Flowchart ini menggambarkan fungsi dengan parameter label, yang memastikan teks tidak boleh kosong. Di dalam perulangan while True, program meminta input dengan input(label).strip(), lalu mengecek apakah teks == ''.

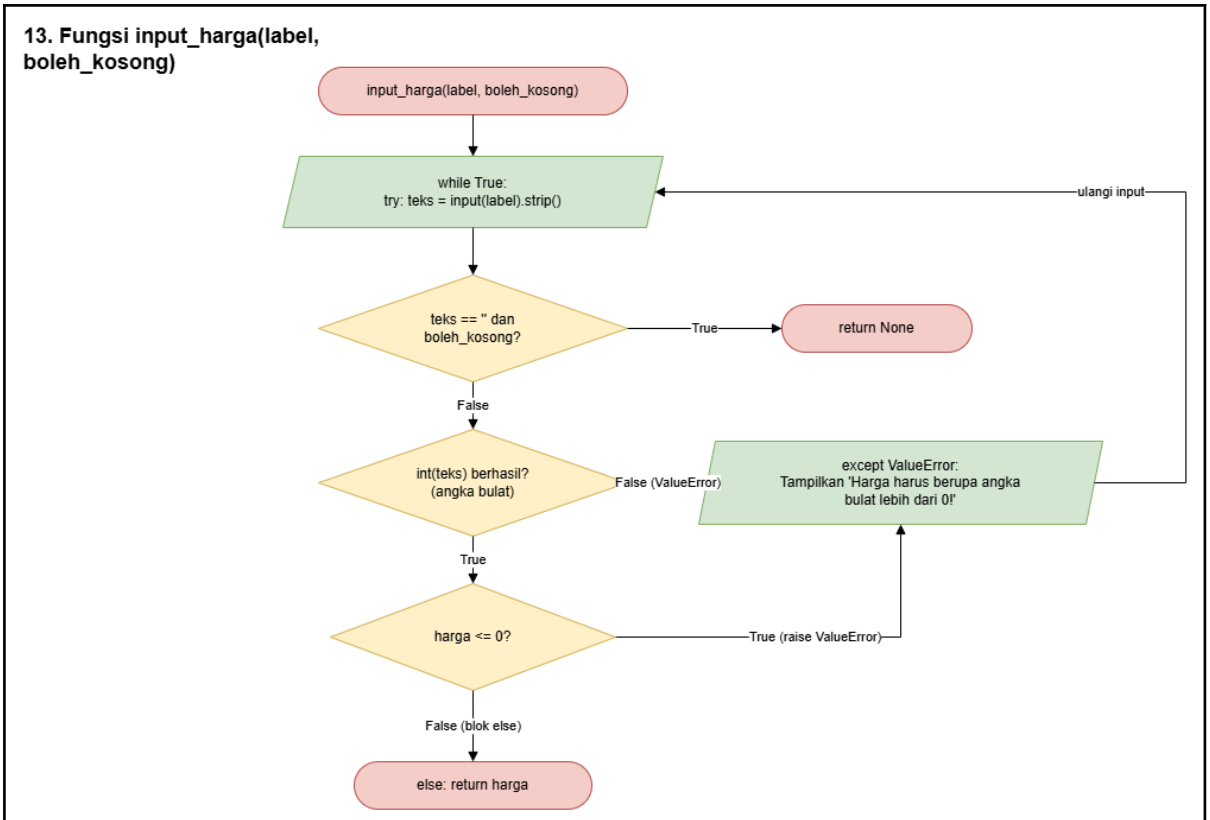
- Jika True, program melakukan raise ValueError('Input tidak boleh kosong!'), ditangkap oleh except ValueError as e yang mencetak pesan, lalu input diulang.
- Jika False, blok else dijalankan dan fungsi mengembalikan teks.



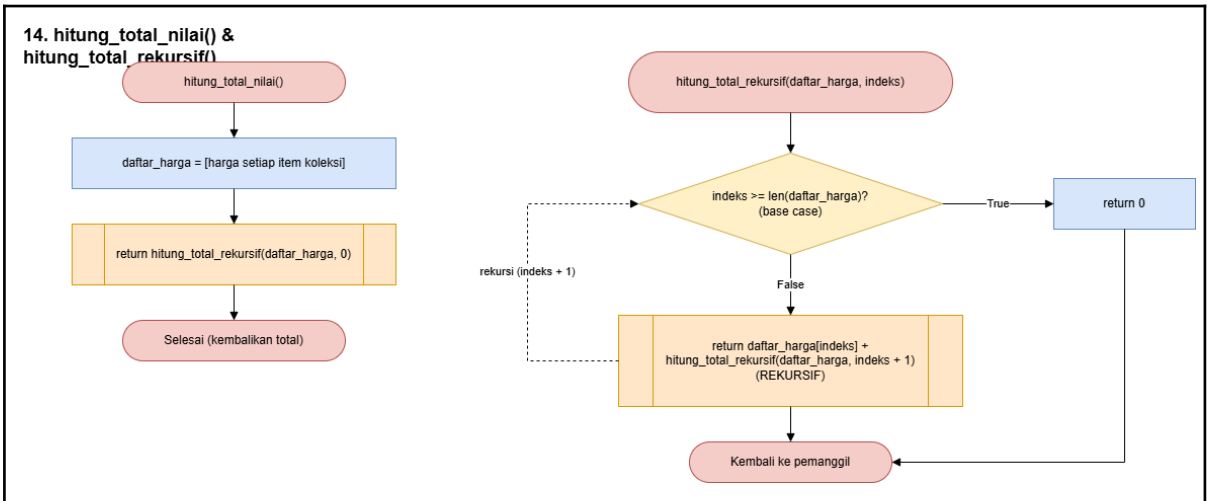
Gambar 1.12 Fungsi Input Harga

Flowchart ini menggambarkan fungsi dengan dua parameter, yang memastikan harga berupa angka bulat lebih dari 0. Program meminta input di dalam while True.

- Jika input kosong dan boleh_kosong bernilai True, fungsi mengembalikan None. Ini dipakai saat mengubah koleksi.
- Jika tidak, program mencoba int(teks). Jika gagal (ValueError), pesan “Harga harus berupa angka bulat lebih dari 0!” ditampilkan dan input diulang.
- Jika berhasil, program mengecek harga <= 0. Jika True, program melakukan raise ValueError, yang juga berujung pada pesan error dan pengulangan input.
- Jika False, blok else dijalankan dan fungsi mengembalikan harga.



Gambar 1.13 Fungsi Input Harga

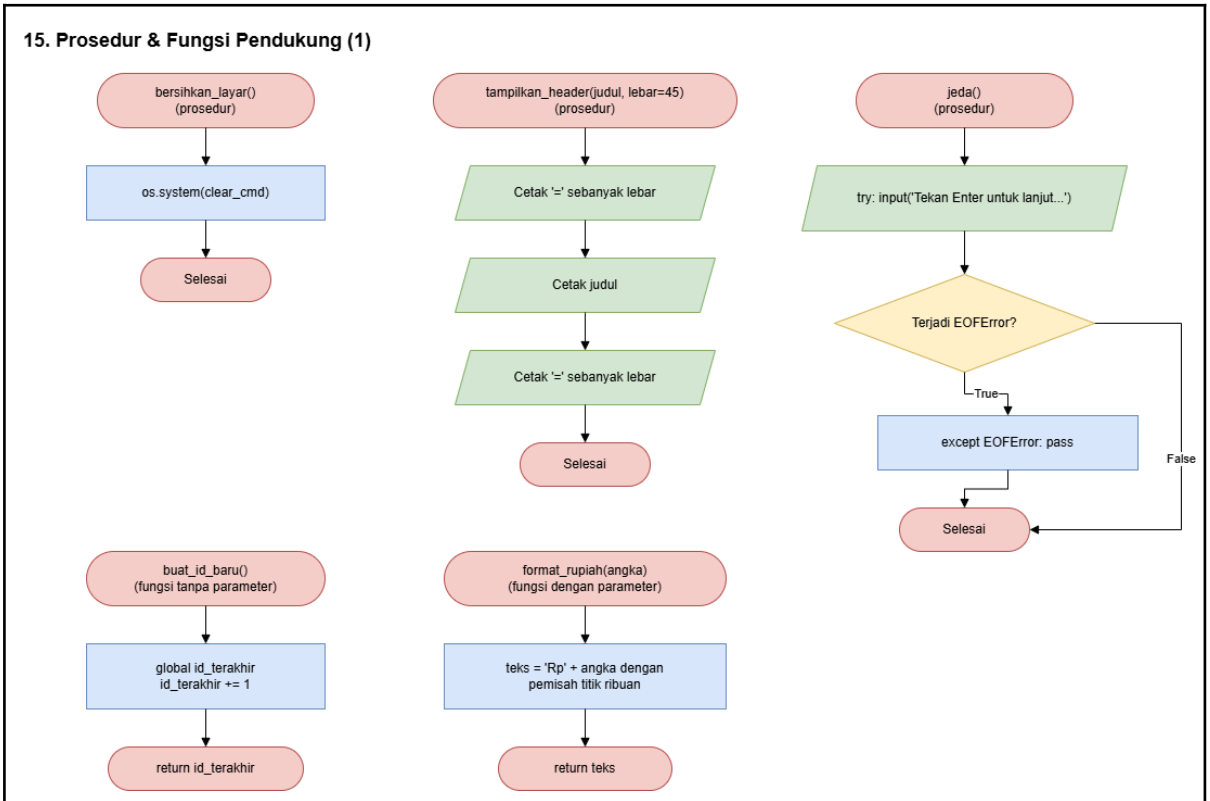


Gambar 1.14 Fungsi Hitung total nilai dan total rekursif

Flowchart ini menggambarkan perhitungan total nilai koleksi. Fungsi hitung_total_nilai() (tanpa parameter) mengumpulkan semua harga koleksi ke daftar_harga,

lalu memanggil hitung_total_rekursif(daftar_harga, 0) dan mengembalikan totalnya. Fungsi hitung_total_rekursif(daftar_harga, indeks) bersifat rekursif.

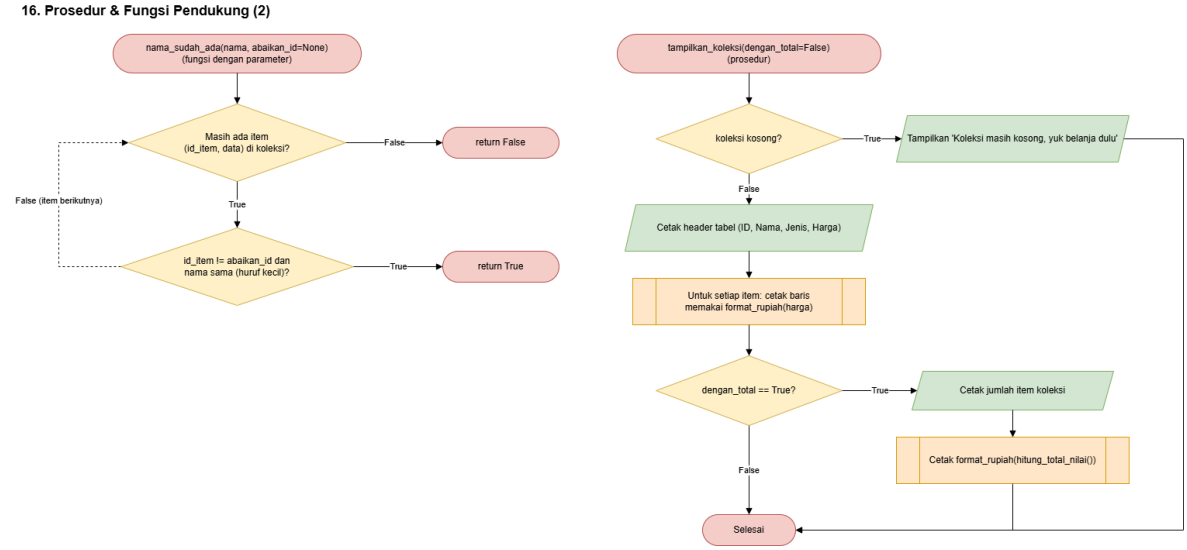
- Base case: jika indeks >= len(daftar_harga), fungsi mengembalikan 0.
- Recursive case: jika belum, fungsi mengembalikan daftar_harga[indeks] ditambah hasil pemanggilan dirinya sendiri dengan indeks + 1.



Gambar 1.15 Prosedur dan fungsi pendukung (1)

- bersihkan_layar() adalah prosedur yang menjalankan os.system(clear_cmd) untuk membersihkan layar.
- tampilkan_header(judul, lebar=45) adalah prosedur yang mencetak garis = sepanjang lebar, lalu judul, lalu garis = lagi.
- jeda() adalah prosedur yang menunggu pengguna menekan Enter di dalam try. Jika terjadi EOFError, except menjalankan pass supaya program tidak crash.
- buat_id_baru() adalah fungsi tanpa parameter yang menaikkan variabel global id_terakhir sebanyak 1 dengan kata kunci global, lalu mengembalikan nilainya.

- format_rupiah(angka) adalah fungsi dengan parameter yang mengubah angka menjadi teks berformat rupiah, misalnya 15000 menjadi Rp15.000, lalu mengembalikannya.



Gambar 1.16 Prosedur dan fungsi pendukung (2)

- nama_sudah_ada(nama, abaikan_id) adalah fungsi dengan parameter yang mengecek nama kembar di koleksi. Fungsi ini memeriksa tiap item. Jika id_item berbeda dari abaikan_id dan namanya sama (tanpa membedakan huruf besar kecil), fungsi mengembalikan True. Jika tidak, fungsi lanjut ke item berikutnya. Jika semua item sudah diperiksa, fungsi mengembalikan False.
- tampilkan_koleksi(dengan_total=False) adalah prosedur yang menampilkan tabel koleksi. Jika koleksi kosong, tampil pesan “Koleksi masih kosong, yuk belanja dulu”. Jika tidak, program mencetak header tabel dan setiap item dengan format_rupiah(harga). Jika dengan_total bernilai True, program juga mencetak jumlah item dan total nilai dari format_rupiah(hitung_total_nilai()).

2. Deskripsi Singkat Program

Program “Koleksi Gantungan Kunci” dibuat untuk membantu pengelolaan data koleksi gantungan kunci secara sederhana dan terstruktur melalui tampilan menu di terminal. Dengan program ini, data koleksi tidak perlu dicatat secara manual karena dapat disimpan, dilihat, diubah, dan dihapus dengan mudah.

Fungsi utama program ini terbagi berdasarkan hak akses pengguna. Admin dapat menambah, melihat, mengubah, dan menghapus data koleksi, serta melihat jumlah koleksi dan total nilainya. Pengguna biasa hanya dapat melihat daftar koleksi tanpa bisa mengubah datanya. Pengguna baru juga dapat membuat akun sendiri melalui fitur register, dan setiap akun harus login terlebih dahulu sebelum bisa mengakses koleksi.

3. Source Code

A. Variabel Global

Data program disimpan di variabel global agar bisa dipakai oleh semua fungsi dan prosedur. users menyimpan akun beserta role-nya, koleksi menyimpan data gantungan kunci, id_terakhir mencatat ID terakhir, clear_cmd menyesuaikan perintah bersihkan layar dengan sistem operasi, dan MAKS_PERCOBAAN menentukan batas percobaan login.

```
  users = {
    "admin": {"password": "admin123", "role": "Admin"},
    "herul": {"password": "herul123", "role": "User"},
}
koleksi = {
    1: {"nama": "Si Kucing Hoki", "jenis": "Hewan", "harga": 15000},
    2: {"nama": "Dino Imut", "jenis": "Hewan", "harga": 20000},
    3: {"nama": "Sepatu Mini", "jenis": "Miniatur", "harga": 12000},
}
id_terakhir = 3
clear_cmd = "cls" if os.name == "nt" else "clear"
MAKS_PERCOBAAN = 3
```

B. Prosedur (tanpa nilai kembalian)

Prosedur menjalankan sekumpulan perintah tanpa return. bersihkan_layar() adalah prosedur tanpa parameter, sedangkan tampilkan_header(judul, lebar) adalah prosedur dengan parameter. Keduanya dipakai berulang di setiap menu sehingga kode tidak perlu ditulis ulang.

```
def bersihkan_layar():
    os.system(clear_cmd)

def tampilkan_header(judul, lebar=45):
    print("=" * lebar)
    print(f"  {judul}")
    print("=" * lebar)
```

C. Fungsi tanpa parameter

Fungsi ini tidak membutuhkan masukan dari luar dan mengembalikan nilai dengan return. buat_id_baru() menaikkan variabel global id_terakhir (memakai kata kunci global), sedangkan hitung_total_nilai() menghitung total harga seluruh koleksi.

```
def buat_id_baru():
    global id_terakhir
    id_terakhir += 1
    return id_terakhir

def hitung_total_nilai():
    daftar_harga = [data["harga"] for data in koleksi.values()]
    return hitung_total_rekursif(daftar_harga, 0)
```

D. Fungsi dengan parameter

Fungsi ini menerima nilai dari pemanggilnya. format_rupiah(angka) mengubah angka menjadi format rupiah bertitik (contoh: 15000 menjadi Rp15.000), sedangkan nama_sudah_ada(nama, abaikan_id) mengecek nama kembar tanpa membedakan huruf besar dan kecil. Parameter abaikan_id dipakai saat mengubah data agar nama miliknya sendiri tidak dianggap kembar.

```
def format_rupiah(angka):
    return "Rp" + f"{angka:,}".replace(",", ".")

def nama_sudah_ada(nama, abaikan_id=None):
    for id_item, data in koleksi.items():
        if id_item != abaikan_id and data["nama"].lower() == nama.lower():
            return True
    return False
```

E. Error handling pada input

Error handling memakai try, except, dan else agar program tidak berhenti saat pengguna salah input. raise dipakai untuk membuat error secara manual, dan input diulang sampai benar. Pada input_harga(), input yang bukan angka memicu ValueError otomatis, sedangkan harga kurang dari atau sama dengan 0 memicu raise ValueError.

```
def input_harga(label, boleh_kosong=False):
    while True:
        try:
            teks = input(label).strip()
            if teks == "" and boleh_kosong:
                return None
            harga = int(teks)
            if harga <= 0:
                raise ValueError("Harga harus lebih dari 0!")
        except ValueError:
            print("Harga harus berupa angka bulat lebih dari 0!")
        else:
            return harga
```

F. Error handling pada pemilihan ID

Fungsi pilih_id() menangani dua jenis error. ValueError muncul jika ID bukan angka, dan KeyError muncul jika ID tidak ada di koleksi. Jika tidak ada error, blok else mengembalikan ID tersebut. Jika ada error, fungsi mengembalikan None.

```
def pilih_id():
    try:
        nomor = int(input("Masukkan ID: ").strip())
        if nomor not in koleksi:
            raise KeyError(nomor)
    except ValueError:
        print("ID harus berupa angka!")
    except KeyError:
        print("ID tidak ditemukan di koleksi!")
    else:
        return nomor
    return None
```

G. Error handling pada program utama

Seluruh program dibungkus try, except, dan finally. Jika pengguna menekan Ctrl+C (KeyboardInterrupt) atau input terputus (EOFError), program keluar dengan pesan yang rapi dan tidak menampilkan error. Blok finally selalu dijalankan, baik terjadi error maupun tidak.

```
try:
    main()
except KeyboardInterrupt:
    print("\nProgram dihentikan paksa (Ctrl+C). Sampai jumpa!")
except EOFError:
    print("\nInput terputus. Program ditutup.")
finally:
    print("Program selesai.")
```

H. Fungsi rekursif untuk menghitung total

Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri. hitung_total_rekursif() menjumlahkan harga satu per satu dengan menaikkan indeks. Base case-nya adalah saat indeks melewati panjang daftar, yaitu mengembalikan 0.

```
def hitung_total_rekursif(daftar_harga, indeks):
    if indeks >= len(daftar_harga):
        return 0
    return daftar_harga[indeks] + hitung_total_rekursif(daftar_harga, indeks
+ 1)
```

I. Login dengan batas percobaan (rekursif)

Pengguna hanya diberi 3 kali kesempatan memasukkan username dan password yang benar. Jika salah, fungsi memanggil dirinya sendiri dengan sisa - 1. Jika sisa mencapai 0 (base case), fungsi mengembalikan None yang berarti login gagal. Jika berhasil, fungsi mengembalikan username dan role.

```
def proses_login(sisa):
    if sisa == 0:
        return None
    username = input("Username: ").strip()
    password = input("Password: ").strip()
    akun = users.get(username)
    if akun is not None and akun["password"] == password:
        return username, akun["role"]
    print(f"Username atau Password anda salah! Sisa percobaan anda: {sisa -
1}")
    return proses_login(sisa - 1)
```

J. Pembagian hak akses (Admin dan User)

Setelah login berhasil, prosedur login() memeriksa role akun. Admin diarahkan ke menu_admin() untuk pengelolaan lengkap, sedangkan User hanya diarahkan ke menu_user() untuk melihat koleksi.

```
def login():
    bersihkan_layar()
    tampilkan_header("LOGIN")
    hasil = proses_login(MAKS_PERCOBAAN)
    if hasil is None:
        print("Mohon maaf, anda gagal login, silahkan kembali ke menu
utama")
        jeda()
        return
    user_aktif, role_aktif = hasil
    if role_aktif == "Admin":
        menu_admin()
    else:
        menu_user(user_aktif)
```

K. Register akun baru dengan validasi

Username tidak boleh kosong, tidak boleh mengandung spasi, dan tidak boleh kembar. Password minimal 4 karakter. Akun baru otomatis berstatus User.

```
if username_baru == "":
    print("username tidak boleh kosong!")
elif " " in username_baru:
    print("username tidak boleh mengandung spasi!")
elif sudah_dipakai:
    print("Username telah di pakai, cari yang lain ya!")
else:
    password_baru = input("Password baru: ").strip()
    if len(password_baru) < 4:
        print("Password minimal 4 karakter!")
    else:
        users[username_baru] = {"password": password_baru, "role": "User"}
        print(f"Akun '{username_baru}' berhasil dibuat! silahkan login")
```

L. Tambah koleksi dengan validasi input

Nama dan jenis tidak boleh kosong, nama tidak boleh kembar, dan harga harus berupa angka lebih dari 0. Validasi dilakukan oleh fungsi input_teks_wajib(), nama_sudah_ada(), dan input_harga(). ID baru dibuat oleh buat_id_baru().

```
def tambah_koleksi():
    bersihkan_layar()
    tampilkan_header("TAMBAH GANTUNGAN KUNCI")
    while True:
        nama = input_teks_wajib("Nama gantungan kunci: ")
        if nama_sudah_ada(nama):
            print("Nama itu sudah di dalam koleksi")
        else:
            break
    jenis = input_teks_wajib("Jenis gantungan kunci: ")
    harga = input_harga("Harga gantungan kunci: ")
    id_baru = buat_id_baru()
    koleksi[id_baru] = {"nama": nama, "jenis": jenis, "harga": harga}
    print(f"YEYY! '{nama}' berhasil masuk ke koleksi")
```

M. Lihat koleksi dalam tabel dengan total nilai

Data ditampilkan dalam tabel rapi dengan harga berformat rupiah. Jika parameter dengan_total bernilai True (dipakai pada menu Admin), prosedur juga menampilkan jumlah koleksi dan total nilainya.

```
def tampilkan_koleksi(dengan_total=False):
    if len(koleksi) == 0:
        print("Koleksi masih kosong, yuk belanja dulu")
        return
    print(f"{'ID':<4}{'Nama':<22}{'Jenis':<14}{'Harga':>12}")
    print("-" * 52)
    for id_item, data in koleksi.items():

print(f"{id_item:<4}{data['nama']:<22}{data['jenis']:<14}{format_rupiah(data
['harga']):>12}")
    if dengan_total:
        print("-" * 52)
        print(f"Total Koleksi: {len(koleksi)} buah")
        print(f"Total nilai  : {format_rupiah(hitung_total_nilai())}")
```

N. Ubah koleksi

Admin memilih data berdasarkan ID dari pilih_id(). Isian yang dikosongkan tidak mengubah data lama. Nama baru dicek dengan nama_sudah_ada() dan harga baru dengan input_harga() (boleh kosong).

```
idx = pilih_id()
if idx is None:
```

```
    return

nama_baru = input(f"Nama baru  [{koleksi[idx]['nama']}]: ").strip()
if nama_baru != "":
    if nama_sudah_ada(nama_baru, abaikan_id=idx):
        print("Nama itu sudah dipakai, nama tidak diubah")
    else:
        koleksi[idx]["nama"] = nama_baru

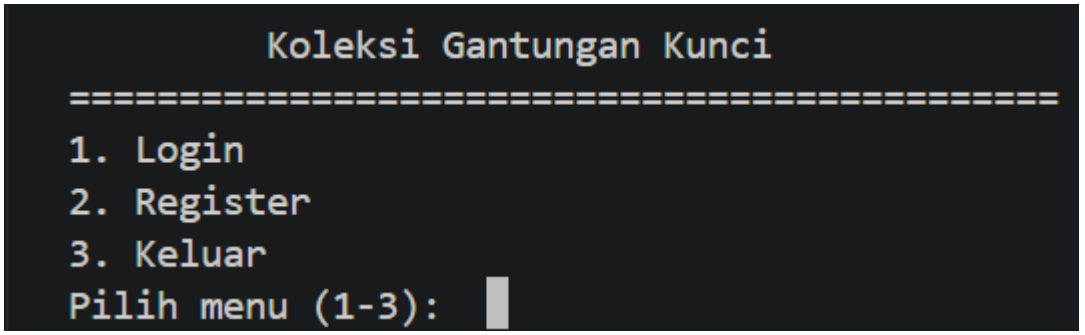
harga_baru = input_harga(f"Harga baru [{koleksi[idx]['harga']}]: ",
boleh_kosong=True)
if harga_baru is not None:
    koleksi[idx]["harga"] = harga_baru
```

O. Hapus koleksi dengan konfirmasi

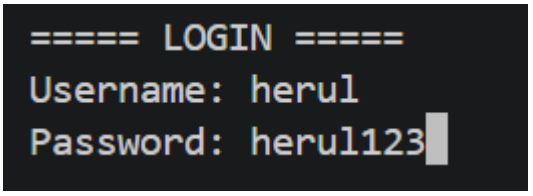
Sebelum data dihapus, program meminta konfirmasi supaya tidak terhapus secara tidak sengaja.

```
yakin = input(f"Yakin hapus '{koleksi[idx]['nama']}'? (y/n):
").strip().lower()
if yakin == "y":
    terhapus = koleksi.pop(idx)
    print(f"'{terhapus['nama']}' sudah di hapus, babayy~")
else:
    print("gajadi di hapus, aman kok!")
```

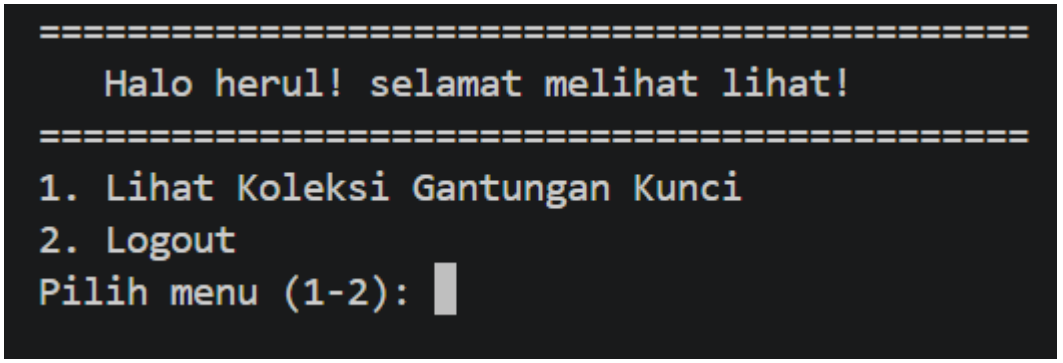
4. Hasil Output



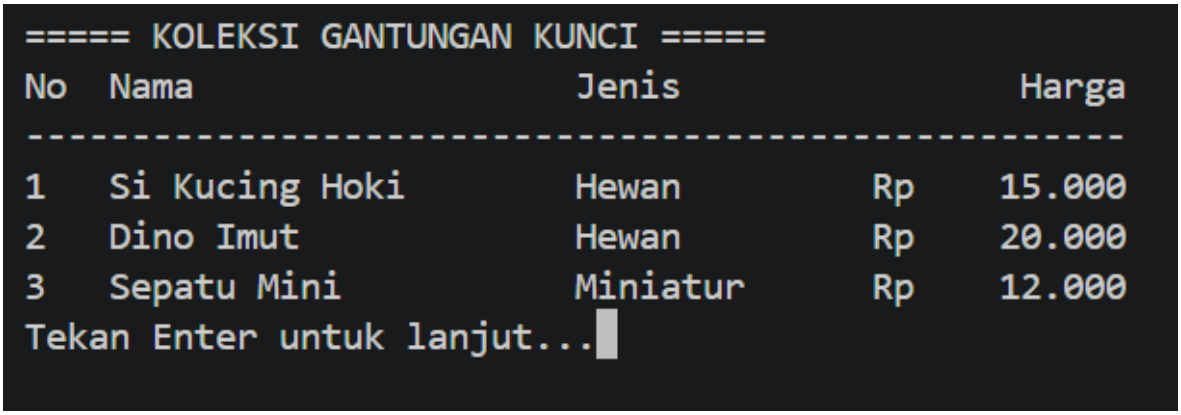
4.1 Ouput Menu Utama



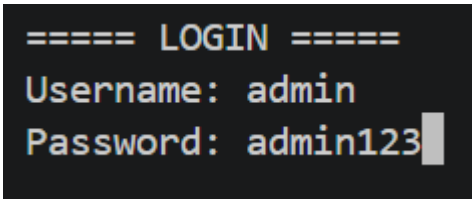
4.2 Ouput Menu Login (User)



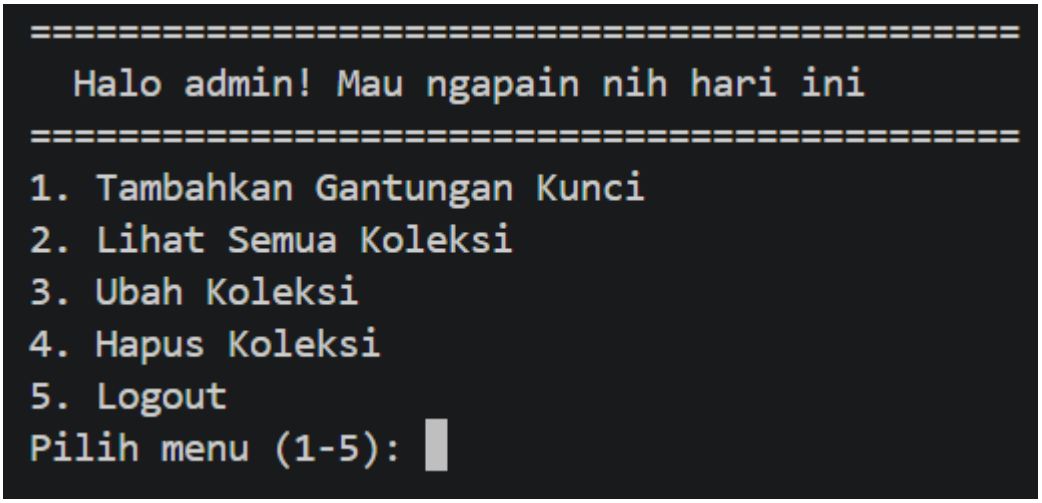
4.3 Output Menu User



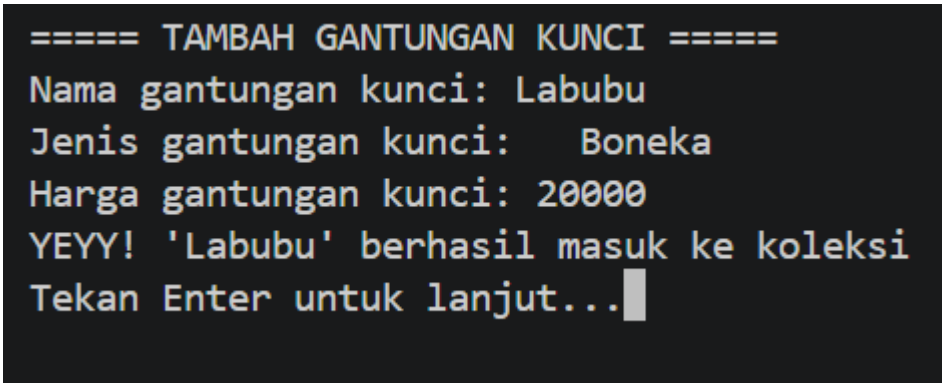
4.4 Output Menu Tampilan Koleksi



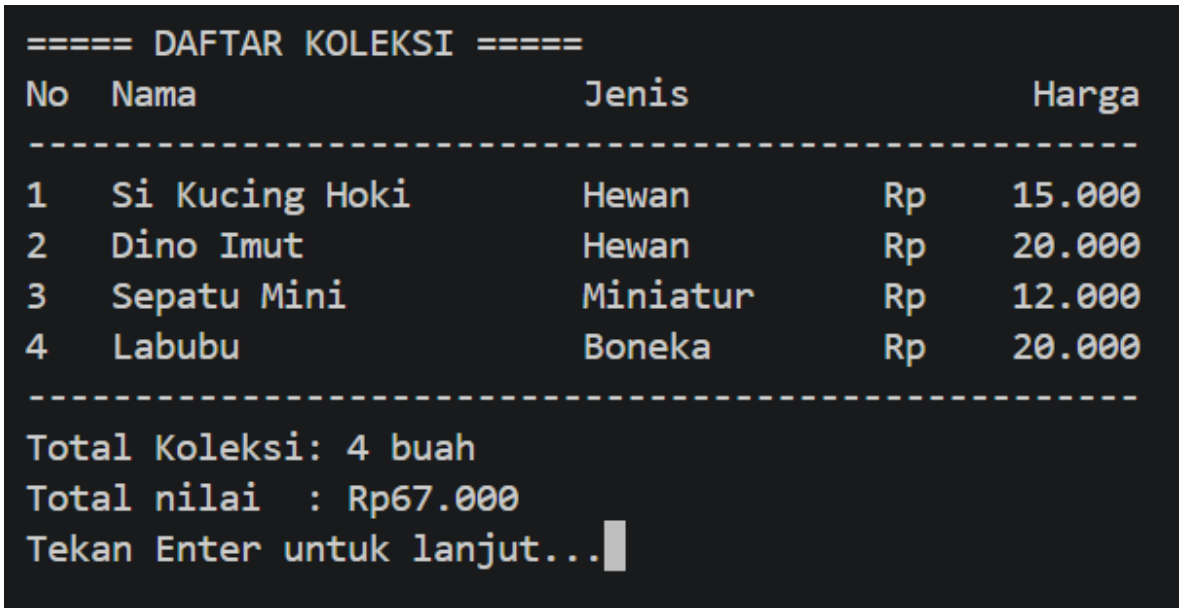
4.5 Output Menu Login Admin



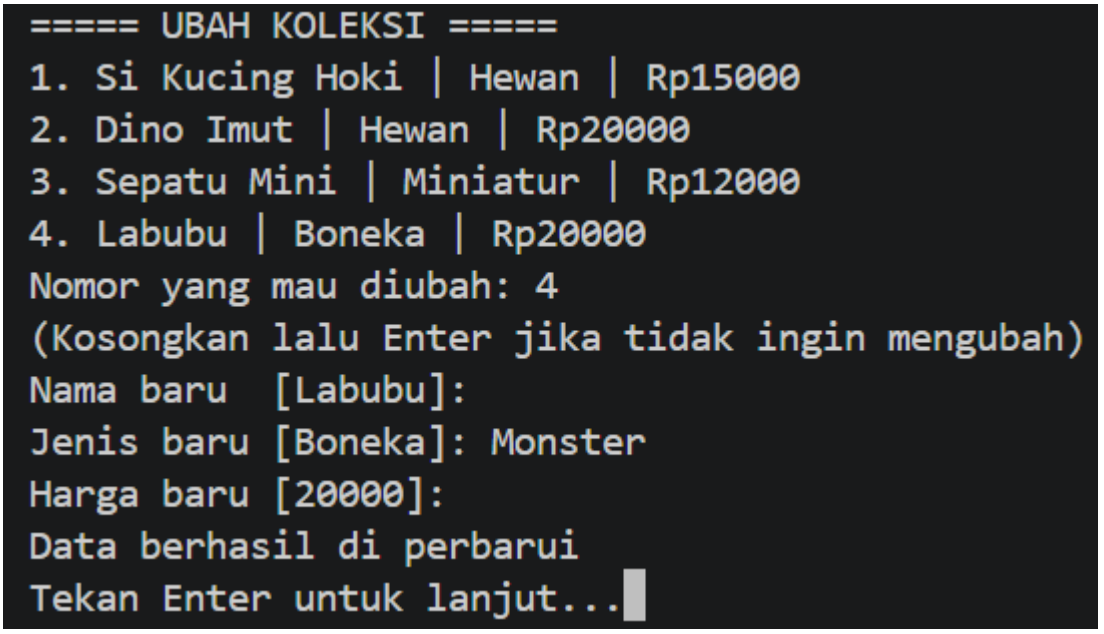
4.6 Output Menu Admin



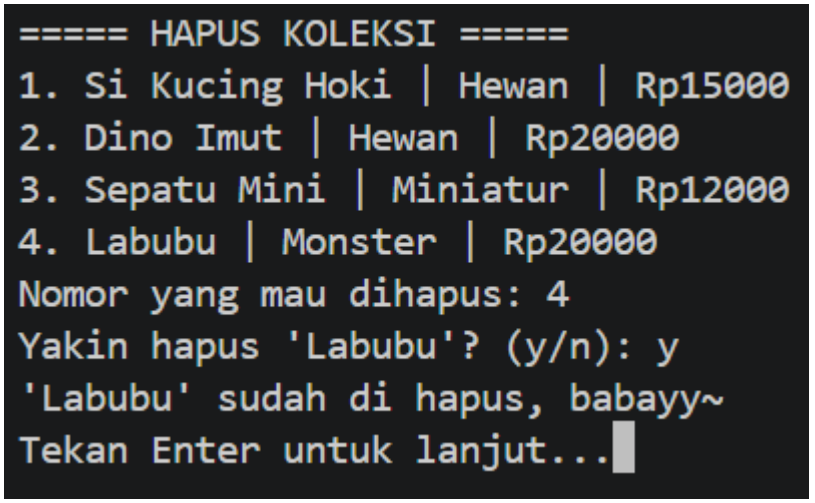
4.7 Output Menambahkan gantungan kunci



4.8 Output Menampilkan koleksi (admin)



4.9 Output Mengubah koleksi



4.10 Output Menghapus koleksi

```
===== REGISTER AKUN BARU =====
Username baru: falih
Password baru: falih123
Akun '{username_baru}' berhasil dibuat! silahkan login
Tekan Enter untuk lanjut...
```

4.10 Output Registrasi Akun baru

5. Langkah-langkah Git

5.1 Git Add

Git add digunakan untuk memasukkan file atau perubahan ke staging area sebelum disimpan ke dalam commit

```
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026> git add .
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026>
```

5.1 Git Add

5.2 Git Commit

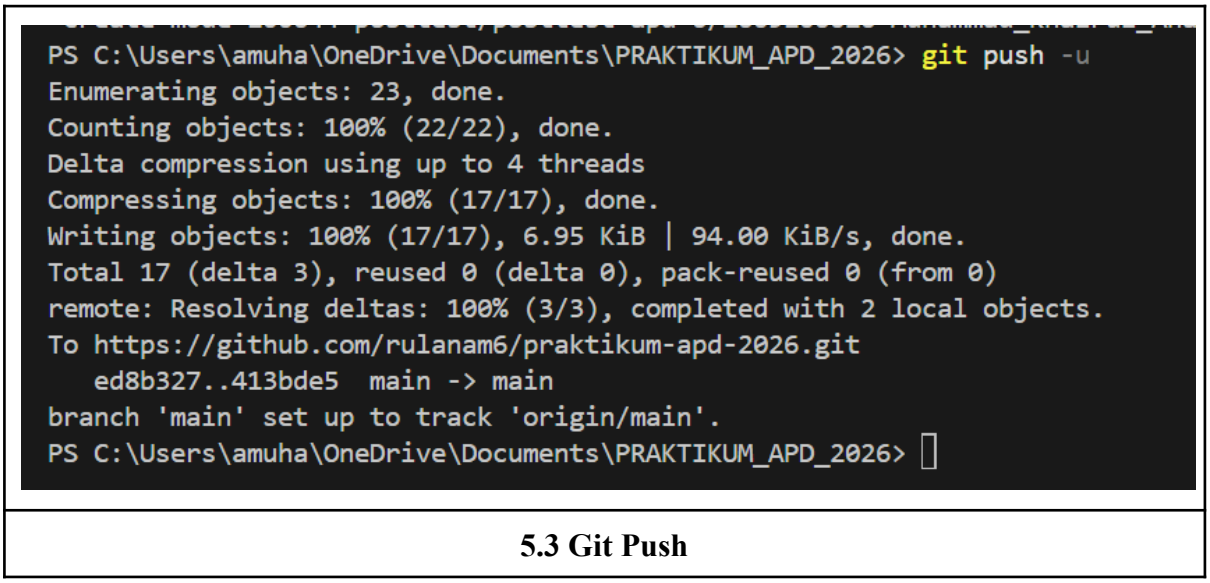
Git commit digunakan untuk menyimpan perubahan yang sudah masuk staging area ke dalam repository lokal. Commit biasanya diberi pesan untuk menjelaskan perubahan yang dilakukan

```
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026> git commit -m "POSTTEST 7"
[main ae87cd1] POSTTEST 7
 7 files changed, 426 insertions(+)
 create mode 100644 kelas/A2_Pertemuan8.py
 create mode 100644 posttest/posttest-apd-7/2609106026-Muhammad_Khairul_Anaam-PT-7.py
 create mode 100644 proyek_akademik/__pycache__/autentikasi.cpython-314.pyc
 create mode 100644 proyek_akademik/__pycache__/mahasiswa.cpython-314.pyc
 create mode 100644 proyek_akademik/autentikasi.py
 create mode 100644 proyek_akademik/mahasiswa.py
 create mode 100644 proyek_akademik/main.py
PS C:\Users\amuha\OneDrive\Documents\PRAKTIKUM_APD_2026>
```

5.2 Git Commit

5.3 Git Push

Git push digunakan untuk mengirim commit dari repository lokal ke repository remote, misalnya Github.



5.3 Git Push